Diagnosing and Repairing Multi-Step Reasoning Failures in Numerical Question Answering over Financial Reports: Final Report

Sameeksha Keshav
Dept. of Computer Science Engineering
North Carolina State University
skeshav2@ncsu.edu

Abstract

I study why retriever–generator models fail at numerical question answering over financial reports (FinQA), and test one repair. I reimplemented a FinQANet-style retriever and pointer generator from scratch together with a DSL interpreter validated on all 8281 questions, and built a multi-axis error taxonomy. Over 4 training runs with `roberta-base` the generator reaches $60.6 \pm 0.7\%$ test execution accuracy (published: 61.2 with RoBERTa-large). The taxonomy shows that retrieval is imperfect, particularly for hybrid table-and-text questions, but is not the bottleneck: giving the generator the gold evidence changes test accuracy by under 0.5 points. The errors are in the generated program: 71% of the failures on programs with 3+ steps are programs that stop too early, and on one-step questions wrong document operands and over-long programs dominate. As a targeted repair I added beam search with execution-guided reranking. The correct answer is among the top 16 beams for 80.3% of test questions, yet reranking recovers only 61.9–62.1% against 61.5% for beam top-1; the gain is small and not statistically significant. Voting across independently trained models helps more (62.4% with three models), which indicates that a single model's candidates share systematic errors. I did not carry out the planned MultiHiertt check. Code: [**TODO:** GitHub link].

1 Introduction

Background. Analysts answer quantitative questions from filings ("what was the change in operating margin?") by finding numbers scattered over tables and prose and combining them with the right arithmetic. FinQA [Chen et al., 2021] turns this into a benchmark: given an earnings report, produce an executable *program* over the evidence whose result is the answer.

Existing approaches. The FinQA paper introduced FinQANet, a two-stage *retriever–generator*: a cross-encoder selects the supporting facts, and a decoder writes the program, copying numbers from them. Later work keeps this design and improves its parts: DyRRen [Li et al., 2023] re-ranks the retrieved facts at each generation step, and APOLLO [Sun et al., 2022] changes the training objectives of retriever and generator. I restrict this project to this family of non-LLM models, which is small enough to retrain and instrument fully.

Limitations. These systems are compared by aggregate accuracy. A wrong answer can come from missing evidence, a wrong operator structure, a wrong operand or an arithmetic slip, and each calls for a different fix, but published work does not say which of them dominates, so it is unclear which component an improvement should target.

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

Challenges. Failures of the two stages are entangled: a program can be wrong because evidence was missed or despite complete evidence, and separating the cases needs a controlled counterfactual. Programs are also hard to compare, since several programs can be correct (commutative operands, independent steps, a constant versus an equal copied number) and an operand value can occur in several cells. Finally, reported differences of a point or less are of the same size as the variation between training runs, so every claim needs a variance estimate.

Approach. I reimplement a FinQANet-style retriever and pointer generator from scratch, together with a program interpreter and metrics validated on all gold programs; build a multi-axis error taxonomy whose failure locus comes from an oracle-evidence counterfactual; classify why wrong programs are wrong; and test one repair suggested by the findings, beam search with execution-guided reranking.

Contributions.

- A from-scratch retriever–generator baseline and DSL interpreter (all 8281 of 8281 gold programs re-execute to the stored answer), reaching $60.6 \pm 0.7\%$ test EA over 4 runs with `roberta-base`.
- A multi-axis error taxonomy with an oracle-evidence locus. Its main finding: retrieval is imperfect (hybrid questions are 14% of the test set but 50% of top-5 retrieval failures) but is not the bottleneck, since gold evidence changes test EA by under 0.5 points.
- A mechanism-level analysis of wrong programs: for 3+ step programs 71% of errors are programs that stop too early; for one-step programs wrong operands and overlong programs dominate.
- A repair study. The correct answer is among the top 16 beams for 80.3% of questions, but beam reranking recovers only 0.4–0.6 EA points (not significant); voting across independently trained models gains 62.4%. This is reported as a negative result with its limits.
- Open code, scripts and tests to regenerate every table in this report.

2 Method

2.1 Dataset and processing

Data. FinQA contains 8281 expert-written questions over S&P 500 earnings reports, split 6251/883/1147 (train/dev/test). A document has pre-text sentences, one table and post-text sentences, on average 30.7 facts and about 839 tokens. Each question has a gold program in a ten-operation DSL (`add`, `subtract`, `multiply`, `divide`, `exp`, `greater`, `table_max/min/sum/average`) and gold supporting facts. Table 1 gives split statistics; Figure 1 shows the distribution over the diagnostic axes used later.

				% by program steps			% by gold evidence		
Split	Questions	Facts/doc	Tokens/doc	1	2	3+	table	text	hybrid
Train	6,251	30.6	841	59	32	8	63	23	14
Dev	883	30.4	834	59	33	8	61	22	16
Test	1,147	31.3	826	57	36	7	62	25	14

Table 1: FinQA statistics computed by `scripts/03_dataset_stats.py`. "Hybrid" means the gold evidence uses both table rows and text sentences.

Processing. All code is my own; only the three JSON files are taken from the public release. (i) *Facts.* Pre- and post-text are concatenated into `text_i`; each table row j becomes `table_j`, with row 0 the header. (ii) *Label-free serialization.* A row is rendered as "the ⟨label⟩ of ⟨column⟩ is ⟨value⟩" for every column, in a fixed format that does not depend on whether the row is gold. The upstream README documents an earlier leak where gold rows were formatted differently from negatives; I avoid this by construction and compare my output with the dataset's stored gold-row text: of 9901 gold table rows, 92.5% match exactly, 7.0% differ only in whitespace, 0.6% are header rows that differ in format, and 0 differ in content. (iii) *Program interpreter.* I wrote a depth-aware parser and an executor for the DSL. Executing every gold program reproduces the stored answer in 8281 of 8281 cases. Getting there required handling cells such as "22% (22 %)", duplicate row labels (the last one wins) and comparison after rounding to five decimals. (iv) *Operand grounding.* The generator copies

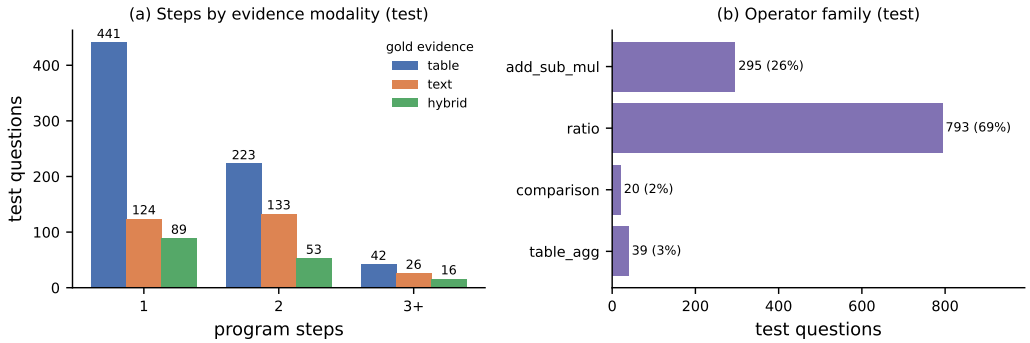


Figure 1: Distribution of questions over the gold-derived axes (program length, evidence modality, operator family).

numbers rather than inventing them, so I extract number and row-label candidates with character offsets. Over all questions 99.9% of gold operands are copyable from the full document, and 99.7% of programs have every operand available. (v) *Dataset quirks.* One gold id (`text_-1`) points to a non-existent fact and is dropped. Several questions have duplicate-valued operands, which is why training uses a marginal loss over duplicates (Sec. 2.2).

2.2 Model design and metrics

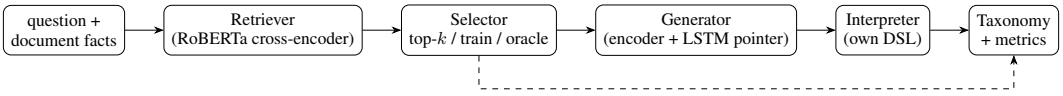


Figure 2: Pipeline. The dashed edge carries the selected evidence into the taxonomy so retrieval and generation errors can be separated.

Retriever. Following FinQANet [Chen et al., 2021], a cross-encoder reads (question, fact) and outputs a relevance score $s(q, f) = \sigma(\mathbf{w}^\top h_{\text{CLS}}(q, f))$ with RoBERTa [Liu et al., 2019] as the encoder, trained with binary cross-entropy over gold facts and sampled negatives. The selector then takes the top k facts. For training the generator I also use *train* (top-$k \cup$ gold) and *oracle* (gold plus the best non-gold up to k) selectors, all in a fixed label-free fact order.

Generator. The selected facts are encoded together with the question. A program is generated token by token by an LSTM decoder with attention. At step t the output distribution is one softmax over reserved tokens (operators, parentheses, constants, step references `#0`–`#7`, end) *and* copy candidates (numbers and row labels in the input):

$$p(y_t \mid y_{<t}) = \text{softmax}\big([\, W_r d_t \,;\, \{e_c^\top W_c d_t\}_{c \in \mathcal{C}}\,]\big), \tag{1}$$

where d_t is the decoder state, e_c the encoder representation of candidate c and $\mathcal{C}$ the candidate set. When several candidates share the gold value, the loss marginalizes over them: $\mathcal{L} = -\sum_t \log \sum_{c \in G_t} p(c \mid y_{<t})$, with G_t the set of correct tokens at step t.

Beam search and execution-guided reranking. For the repair I decode with beam search (up to 16 finished programs per question, ranked by summed log-probability) and choose among the candidates by their *executed answers*. *Voting*: programs that execute to the same answer pool their probability mass $\exp\big((\log p - \max \log p)/T\big)$, the answer with the most mass wins and its most probable program is returned; the beam size k and temperature T are chosen on dev. *Learned*: a classifier (logistic regression or gradient boosting, chosen by 5-fold cross-validation on dev) scores each candidate from 38 features (log-probability and rank, vote share and number of distinct answers, program length and shape, answer plausibility, question cues); it is trained on the dev beams, since the generator never trained on dev, and applied once to test. In both cases the answer is chosen by the

reranker and the program by the generator's probability, so program accuracy is not degraded when several programs give the same answer.

Metrics. *Execution accuracy* (EA) compares the executed answer with the gold answer after rounding to five decimals. *Program accuracy* (PA) compares a canonical form of the program: commutative operands are sorted, independent steps are order-normalized, and a constant is treated as equal to a copied number of the same value. This last choice is lenient and is documented and tested; for example `divide(637, const_5)` equals `divide(637, 5)` under PA. The published FinQANet numbers are 61.24 EA and 58.86 PA with RoBERTa-large [Chen et al., 2021].

Error taxonomy. Every test question is tagged on gold-derived axes (steps 1/2/3+, modality table/text/hybrid, operator family, number of gold facts) and, once predictions exist, on prediction-derived axes: *retrieval status* (complete, partial, miss), *operands grounded*, *error type* (non-executable, structure error, operand error, found by comparing predicted and gold *skeletons* with operands masked), and *locus*. The locus uses a counterfactual: rerun the generator with the oracle selector; if the answer becomes correct the error was in retrieval, if it stays wrong it was in generation, and if neither selector is correct it is recorded as both.

3 Results

Where the released retriever loses evidence. The FinQA authors released the retriever's ranked facts. I analyse these test-set rankings with my own taxonomy code, with one caveat: I do not know the exact checkpoint that produced them, so this is an analysis of released outputs, not a reproduction. *Full recall@5* is the share of questions whose entire gold evidence is in the top 5. It is 86.5% overall, against 11.5% for a random top-5. Table 2 and Figure 3 break this down. Table-only questions are retrieved well (95.3%), text-only less so (84.1%), and hybrid questions poorly (51.3% full recall, although 77.2% of their individual gold facts are found). Hybrid questions are 14% of the test set but account for 77 of the 155 top-5 failures (50%). Program length matters much less than evidence modality: recall is 87.3% for one-step and 84.5% for 3+ step programs.

Category (test)	n	Full recall@5	Fact recall@5
All questions	1147	86.5	89.8
Gold evidence modality			
table	706	95.3	96.2
text	283	84.1	85.2
hybrid	158	51.3	77.2
Program steps			
1	654	87.3	90.9
2	409	85.6	87.6
3+	84	84.5	89.8
Operator family			
add_sub_mul	295	83.1	87.5
ratio	793	86.9	90.1
comparison	20	95.0	97.2
table_agg	39	100.0	100.0
Random top-5 (chance)		11.5	

Table 2: Released-retriever top-5 evidence recall on the test set, by taxonomy axis (`scripts/02_retrieval_analysis.py`).

Missing evidence is not always fatal. Of the 155 test questions whose top-5 misses part of the gold evidence, 106 (68%) still have all operands copyable from the retrieved facts. Whether the generator then produces the right program is what the oracle counterfactual below measures.

Trained baseline. I trained both models on a Kaggle T4 with `roberta-base` (not RoBERTa-large as in the published number, so some gap is expected); the checkpoint is chosen by dev EA. The table 3 runs use seed 0; to estimate run-to-run variation I trained the released-retrieval generator 4

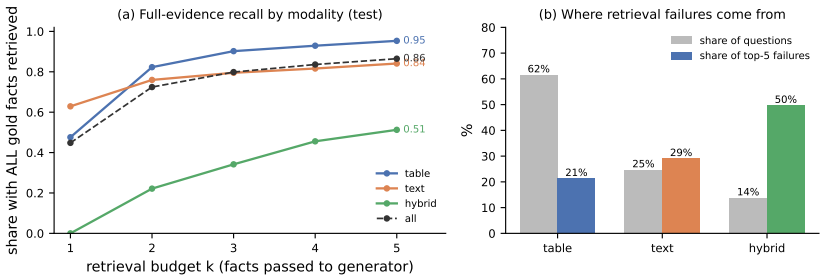


Figure 3: Full evidence recall as the cut-off k grows, split by modality (`scripts/04_plot_retrieval.py`).

times (seeds 0, 1, 2 and a repeat of seed 0; GPU training is not bit-reproducible): test EA $60.6 \pm 0.7\%$ (range 59.6–61.1), PA $58.7 \pm 0.6\%$. My retriever reaches 86.1% full recall@5 on the test set (released: 86.5%), with 93.5% on table, 84.1% on text and 57.0% on hybrid questions (released: 95.3, 84.1, 51.3). Table 3 gives generator accuracy. With the released retrieval the generator reaches 60.5% EA and 58.8% PA on test (dev best 63.2% EA), within a point of the published 61.24/58.86 although the encoder is smaller. My retriever gives 60.1% EA, a 0.4-point difference that is inside the run-to-run variation and the sampling noise of 1,147 test questions, and which I do not interpret.

	Dev		Test	
Setting	EA	PA	EA	PA
Released retriever + top-5 evidence	63.2	61.5	60.5	58.8
Released retriever + gold evidence	64.4	62.9	60.6	58.9
My retriever + top-5 evidence	63.2	61.0	60.1	58.2
My retriever + gold evidence	64.2	62.1	60.2	58.4

Table 3: Generator accuracy (%) with each retriever, using its top-5 facts or the gold facts (oracle). First run only; run-to-run standard deviation is about 0.7 points.

Error taxonomy: retrieval is not the bottleneck. Using the oracle counterfactual from Sec. 2.2, I split the 453 wrong test answers of the released-retrieval run by locus. 356 (78.6%) are generation errors with complete retrieval; 89 (19.6%) have incomplete retrieval but remain wrong when the generator is given the gold evidence; only 8 (1.8%) are fixed by gold evidence. Equivalently, oracle evidence moves test EA from 60.5% to 60.6%. Within the 356 wrong answers with complete retrieval, 203 have a different operator structure from the gold program and 152 have the right structure with a wrong operand. Table 4 shows where accuracy is lowest: programs with 3+ steps (21.4% EA, against 66.8% for one step, with almost all errors structural) and non-table evidence (hybrid 44.9%, text 50.2%, table 68.1%). Hybrid questions are hard for two reasons that this analysis separates: their evidence is retrieved less often, but even when it is, the generator does worse on them.

Category (test)	n	EA	Retr. complete	Structure err.	Operand err.
Gold evidence					
table	706	68.1	95.3	120	105
text	283	50.2	84.1	92	48
hybrid	158	44.9	51.3	57	30
Program steps					
1	654	66.8	87.3	99	117
2	409	58.4	85.6	109	61
3+	84	21.4	84.5	61	5

Table 4: Released-retrieval run on the test set: accuracy, share of questions with complete retrieval, and error types of wrong answers (`scripts/07_taxonomy_report.py`).

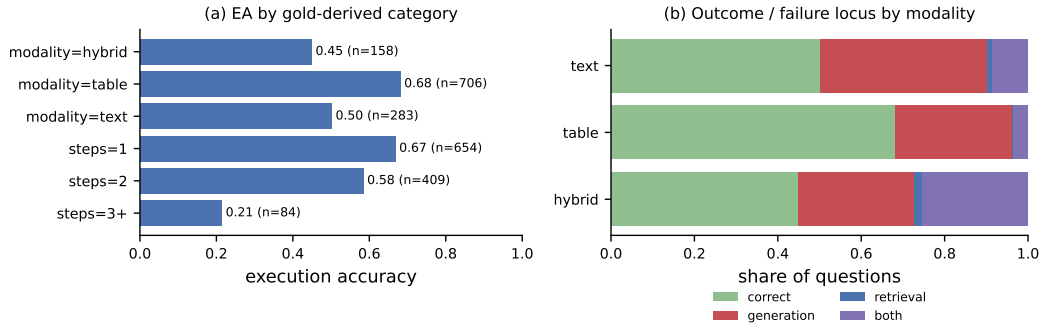


Figure 4: Released-retrieval run, test set: (a) EA by gold-derived category; (b) outcome and failure locus by evidence modality.

How the generator fails. The skeleton-level error types above are coarse, so I classified every wrong program by the first structural difference from the gold program (`mechanisms.py`), pooling the 4 runs (1809 wrong answers). Table 5 shows that the mechanism depends strongly on program length. For 3+ step programs, 71% of the errors are programs with too few steps (too few steps is 23.7% of all errors), so the model stops before the program is complete. For one-step questions, wrong document operands (49%) and programs with too many steps (29%) dominate. Wrong operands are the largest single category overall (36.2%); swapped arguments, wrong constants and wrong step references together are only 7%. A different constant convention for percentages is not the explanation: it accounts for about 2% of errors.

Mechanism (% of wrong answers)	All	1 step	2 steps	3+ steps
not a valid program	0.0	0.0	0.0	0.0
too few steps	23.7	0.0	36.7	71.4
too many steps	18.6	29.3	5.7	15.7
wrong operator (same step count)	14.4	15.6	16.8	3.9
arguments swapped	2.3	2.7	2.7	0.0
wrong constant	3.7	3.2	4.9	2.4
wrong step reference	1.1	0.0	1.9	2.4
wrong document operand	36.2	49.2	31.3	4.3
Wrong answers pooled (n)	1809	883	671	255

Table 5: Why wrong programs are wrong, pooled over 4 runs (test set, released retrieval), by gold program length (`12_phase2_analysis.py`). Columns do not mix: each is the distribution within its group.

Repair: beam search and execution-guided reranking. The analysis suggests the generator often has the right program available but does not rank it first. Beam search tests this. Table 6 and Figure 5 show that the right answer is among the top 16 beams for 80.3% of test questions (top-1: 61.5%, top-4: 73.8%), so a perfect reranker would gain 19 points. The two rerankers capture only a small part of that. Answer voting with dev-selected k=8, T=2.0 reaches 61.9% EA against 61.5% for beam top-1 (and 61.1% for greedy decoding of the same model): 6 questions fixed and 1 broken, exact McNemar $p = 0.12$. The learned reranker (gbm d3) reaches 62.1% EA, 11 fixed and 4 broken, with a bootstrap 95% interval for the gain of [0.0, 1.3] points, which includes zero. Its most important feature by far is the vote share, i.e. the same agreement signal that voting uses. On the 3+ step group (n=84) EA rises from 26.2% to 29.8% (voting) and 29.8% (learned), which is three questions and well within noise.

By contrast, voting over independently trained models gains more: 62.4% with three models and 63.0% with four, against a single-model mean of 60.6%, at the cost of training several models. Any of the three models contains the right answer for 68.9% of questions. The candidates in one beam are therefore highly correlated: they tend to repeat the same mistake, so agreement among them is weak

Test set, released retrieval	EA	PA
Greedy decoding (same model)	61.1	59.3
Beam search (16), top-1	61.5	59.5
+ answer voting over beams (k=8, T=2.0; dev-selected)	61.9	59.8
+ learned reranker (gbm d3; dev-trained)	62.1	60.0
Voting over 3 independently trained models	62.4	–
Oracle: right answer among top-16 beams	80.3	–
Oracle: right answer from any of 3 models	68.9	–

Table 6: Test-set accuracy (%) of decoding and reranking strategies for one generator, and two reference points that use more than one model or the gold answer. Reranking configurations are fixed on dev before looking at test.

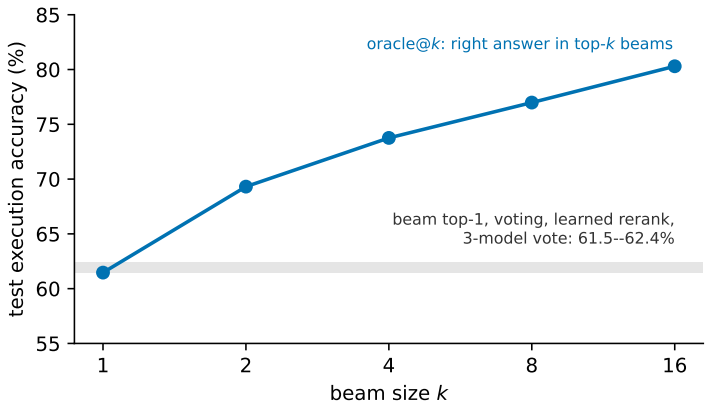


Figure 5: The right answer is usually in the beam (oracle@k) but reranking recovers little of the gap.

evidence of correctness. I did not test why (for example whether the early-stopping errors are shared by all beams), so this is a hypothesis, not a finding.

4 Discussion, Limitations and Scope

What the project shows. (1) In this model family, retrieval quality is not what limits accuracy on FinQA: the released retriever misses evidence on hybrid questions, but a generator with full evidence is barely better. (2) The failures are in the program, and their form depends on length: premature stopping for multi-step programs, wrong operands and overlong programs for short ones. (3) The right program is often in the beam, but neither the model's probabilities nor agreement among beams finds it, and cheap reranking recovers about 0.4–0.6 points. A repair targeting the generator itself (for example length-aware training or decoding) is the natural next step, but I did not implement it.

Limitations. All models use `roberta-base`, not the RoBERTa-large of the published number; 4 runs of one configuration give a rough variance estimate, not a confidence interval; the reranking gains are not statistically significant on 1,147 test questions; the oracle selector supplies gold facts plus the best non-gold facts up to k, so it measures the effect of complete evidence, not of removing all distractors; PA is lenient about constants (Sec. 2.2); the released retrieval has unknown provenance; and only FinQA was used. *Not done:* the MultiHiertt generalization check from the plan, a retrieval-side repair, and larger encoders.

5 Related Work

FinQA and FinQANet [Chen et al., 2021] define the task and the retriever–generator baseline that I reimplement. DyRRen [Li et al., 2023] adds a reranker that conditions on the partially generated

program, and APOLLO [Sun et al., 2022] improves retrieval and generation training objectives; both report accuracy gains but not a failure-mode breakdown. Reranking candidate programs by their execution results follows a general idea in neural program synthesis; here it is applied to FinQA and measured against an oracle. TAT-QA [Zhu et al., 2021], ConvFinQA [Chen et al., 2022] and MultiHiertt [Zhao et al., 2022] are related hybrid-QA benchmarks.

6 Code and Experiment Running Snippets

Code: [**TODO:** GitHub link]. The README lists every file, class and function, and which parts are my own: all of `src/`, `scripts/` and `tests/`. Only the FinQA data files and the pretrained RoBERTa weights are taken from outside. The released retriever rankings come from the FinQA release.

```
pip install -r requirements.txt
bash scripts/00_download_data.sh        # FinQA json files
python -m pytest -q                     # 68 tests
python scripts/01_validate_dsl.py       # gold programs re-execute

# CPU analyses (Tables 1-3, Fig. 1-3)
python scripts/01b_validate_preprocessing.py
python scripts/02_retrieval_analysis.py
python scripts/03_dataset_stats.py
python scripts/04_plot_retrieval.py

# GPU (Kaggle T4; notebooks/ has the exact runs)
python scripts/05_train_retriever.py --model roberta-base --out runs/retriever
python scripts/06_train_generator.py --model roberta-base \
       --retrieval released --seed 0 --out runs/gen_released
python scripts/07_taxonomy_report.py --pred runs/gen_released/pred_test.json \
       --oracle runs/gen_released/pred_test_oracle.json --name released

# Phase 2: beam search and reranking
python scripts/09_beam_decode.py --ckpt runs/gen_beam --beam 16      # GPU
python scripts/10_rerank_eval.py --beams runs/gen_beam --name released
python scripts/11_learned_rerank.py --beams runs/gen_beam --name released
python scripts/12_phase2_analysis.py     # error mechanisms, voting, figure
```

References

Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan Routledge, and William Yang Wang. FinQA: A dataset of numerical reasoning over financial data. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3697–3711, Online and Punta Cana, Dominican Republic, 2021. Association for Computational Linguistics.

Zhiyu Chen, Shiyang Li, Charese Smiley, Zhiqiang Ma, Sameena Shah, and William Yang Wang. ConvFinQA: Exploring the chain of numerical reasoning in conversational finance question answering. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 6279–6292. Association for Computational Linguistics, 2022.

Xiao Li, Yin Zhu, Sichen Liu, Jiangzhou Ju, Yuzhong Qu, and Gong Cheng. DyRRen: A dynamic retriever-reranker-generator model for numerical reasoning over tabular and textual data. In *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence*, pages 13139–13147. AAAI Press, 2023.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

Jiashuo Sun, Hang Zhang, Chen Lin, Xiangdong Su, Yeyun Gong, and Jian Guo. APOLLO: An optimized training approach for long-form numerical reasoning. *arXiv preprint arXiv:2212.07249*, 2022.

Yilun Zhao, Yunxiang Li, Chenying Li, and Rui Zhang. MultiHiertt: Numerical reasoning over multi hierarchical tabular and textual data. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6588–6600. Association for Computational Linguistics, 2022.

Fengbin Zhu, Wenqiang Lei, Youcheng Huang, Chao Wang, Shuo Zhang, Jiancheng Lv, Fuli Feng, and Tat-Seng Chua. TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 3277–3287. Association for Computational Linguistics, 2021.